

Prova com consulta. Duração: 1h30m.

Primeiro Mini-Teste

```
prog foo (n, t) {
1: push 0
2: dup
3: store r
4: store i
5: load i
6: load n
7: ifg 17
8: load r
9: load i
10: add
11: store r
12: load i
13: load t
14: add
15: store i
16: goto 5
17: load r
18: return
}
```

Grupo 1. (9 valores)

Considere o código do exemplo apresentado à esquerda. Suponha que o mesmo inclui todas as instruções de uma máquina baseada em pilha e que n , t , e as letras a seguir às instruções *load* e *store* identificam variáveis. Assuma que as variáveis podem ser identificadas por uma letra do alfabeto.

1.a) [1v] Apresente as definições dos símbolos terminais para a linguagem apresentada.

1.b) [2v] Desenhe um DFA para a análise lexical.

1.c) [3v] Considerando as definições de símbolos terminais da alínea a), apresente uma gramática livre de contexto (CFG – *context free grammar*) que aceite sequências de instruções da máquina. Note que o código apresentado é um exemplo de uma sequência de instruções da máquina.

1.d) [2v] Altere a gramática de forma a aceitar programas nesta linguagem, mas sem a numeração das instruções e usando *labels* para os saltos (condicionais e incondicionais).

1.e) [1v] Podemos considerar que o código apresentado se encontra numa representação de alto-nível? Justifique a resposta dada.

Grupo 2. (11 valores)

Considere a gramática e o exemplo seguintes.

Gramática	Prog	::= "prog" Name "(" Params ")" "{" Stmt "}"	Exemplo	<pre>prog foo (n, t) { r := 0; i := 0; while (i <= n) { r := r+i; i := i+t; }; return r }</pre>
	Params	::= Var {"," Var}		
	Expr	::= Const Var Expr Op Expr		
	Op	::= "+" "<="		
	Stmt	::= Var ":=" Expr Stmt ";" Stmt "if" Expr "then" Stmt "else" Stmt "while" Expr "{" Stmt "}" "return" Var		
	Var	::= "r" "t" "n" "i" "j" "x" "y" "z" "u"		
	Const	::= Digit { Digit }		
	Digit	::= "0" "1" "2" "3" "4" "5" "6" "7" "8" "9"		
	Name	::= Letter { Letter }		
	Letter	::= "a" "b" ... "z"		

2.a) [1v] Apresente uma possível árvore de sintaxe concreta obtida por derivação o mais à esquerda (*leftmost derivation*) para o exemplo.

2.b) [1v] Indique uma possível árvore de sintaxe abstrata (AST – *abstract syntax tree*) para o exemplo.

2.c) [1v] Esta gramática pode ser implementada por um analisador sintático LL(k)? Se sim, indique o valor de k que teria de usar.

2.d) [4v] Apresente uma gramática que defina a mesma linguagem, que não tenha recursividade à esquerda, e que possa ser implementada por um analisador sintático LL(1). (Nota: só precisa de apresentar as produções que tiverem de ser modificadas ou adicionadas)

2.e) [1v] Indique a(s) modificação(ões) que teria de realizar na gramática original para que os programas nesta linguagem pudessem ter zero ou mais parâmetros.

2.f) [1v] Assumindo que as variáveis usadas num programa só podem ser parâmetros ou variáveis locais, indique uma possível tabela de símbolos para este exemplo.

2.g) [2v] Descreva possíveis verificações semânticas a ter em conta para esta linguagem e indique como poderiam ser realizadas essas verificações.

(Fim.)